

CONTENTS

– Abstract	4.1 Feature Overview
1 Introduction	4.2 Technology Stack
1.1 General	4.3 Benefits
1.2 Problem Statement	5 Concept & Methodology
1.3 Objectives	6 Design, Modelling & Architecture
2 Literature Review	7 Implementation
3 Existing System	8 Results & Discussion
3.1 Traditional System	9 Conclusion
3.2 Feature Comparison	10 Future Scope
3.3 Limitations	11 References
4 Proposed System	12 Appendix

RecoverChain

A deterministic decisioning engine for recovering failed revenue — with machine learning kept strictly in the passenger seat.

Domain Revenue recovery / FinTech decisioning

Stack FastAPI · SQLAlchemy · React · XGBoost

Backend tests 216 passing · 88% cov

Code ~15.4k LOC Python · ~3.7k LOC TS/TSX

Iterations 33 commits · ~24 build phases

Report date 2026-09-02

ABSTRACT

Abstract

Businesses that bill customers — subscriptions, invoices, marketplaces, checkouts — lose a persistent slice of revenue not because customers refuse to pay, but because a payment fails, an invoice slips past its due date, or a checkout is abandoned. This *involuntary* leakage is usually chased with blind retry schedules and generic reminder emails that treat every failure alike.

RecoverChain reframes recovery as a **decisioning** problem. Failure events are correlated into *cases*, and every case is driven through one fixed, auditable lifecycle: assess risk → diagnose the root cause → predict the odds of recovery → recommend the next-best action and its expected recoverable value → clear a policy and safety gate → execute in a sandbox → verify the outcome against a settlement source → write an audit record. Each stage is a small deterministic rules engine that produces an explanation a human can read.

A second subsystem, the **Dataset Lab**, lets a user upload an arbitrary CSV, Parquet, or Excel file; it semantically maps columns to canonical fields, runs leakage detection and a minimum-information contract, trains a per-dataset model, and replays the rows through the pipeline to produce real cases. Crucially, every model is **shadow-only**: its probability feeds ranking and prioritisation but is architecturally barred from authorising an action — the deterministic policy engine is the sole authority, and prediction always falls back to a deterministic baseline rather than a silent zero.

The system is delivered as a FastAPI backend (clean domain / application / infrastructure layering, SQLAlchemy + Alembic, `Decimal` money) and a React single-page frontend presenting a ten-step guided workflow plus a gated aggregate report. It runs fully offline with no external service calls. The implementation is validated by **216 automated backend tests (88% line coverage)**, an end-to-end walk-through of the ten-step workflow in a browser, and a hardening pass that added authentication on every mutating route, a request rate limiter, CORS correctness, container and CI security, and a lint / type-check / coverage gate.

Introduction

1.1 General

Any organisation that charges customers on a recurring or deferred basis carries a structural revenue leak. A card expires between billing cycles; a bank declines a debit for insufficient funds; a network timeout drops a transaction; an invoice with net-30 terms is simply forgotten; a shopper fills a cart and closes the tab. None of these are a decision *not* to pay — industry practitioners call this **involuntary churn**, as distinct from voluntary cancellation. Left alone, a meaningful fraction of it never converts back into collected cash.

The established response is **dunning**: a scheduled sequence of automated payment retries and reminder messages. Payment processors ship retry logic (“smart” or “intelligent” retries) that pick retry times statistically; standalone dunning tools send escalating email/SMS sequences; larger finance teams run in-house scripts and manual collections queues. These approaches share a blind spot: they act on a *schedule*, not on a *diagnosis*. A retry fired at a card that expired is wasted. A payment-link email sent to a customer whose bank is rejecting the mandate solves nothing. And when teams do reach for machine learning to prioritise, the model frequently ends up *in the loop* — deciding what runs — which in a financial-actions context is difficult to audit, explain, or safely roll back.

RecoverChain is built around a different premise: **the decision of whether and how to act on a failed payment should be made by a transparent, deterministic rulebook**, with every stage of reasoning recorded, and with statistical models contributing only prioritisation signal that can never, by construction, trigger an action.

1.2 Problem Statement

Design and build a system that, given a stream (or a bulk file) of revenue-failure events, will for each affected account:

1. Correlate related events into a single **case** so that a retry of the same invoice is one case, not two.
2. Produce an **explainable** assessment at every stage — risk level, likely root cause, recovery probability, recommended action, and expected recoverable value — each with the signals that produced it.
3. Pass the recommended action through a **policy and safety gate** that enforces retry caps, cooldowns, contact-frequency limits, an automated-amount ceiling, consent, and hard stop rules, using the account’s *real* prior-attempt history.
4. Execute only what the gate permits, in a **simulated** environment (no live money movement), with idempotency so a replay produces the same record.
5. **Verify** the true outcome independently against a settlement source rather than assuming an executed action succeeded.
6. Accept **arbitrary tabular datasets**, understand their columns, check them for target leakage and sufficiency, train a model on them, and score real rows — without that model ever gaining authority over an action.
7. Run **fully offline**, persist a complete **audit trail**, and be safe to demonstrate and hand over.

1.3 Objectives

#	OBJECTIVE	HOW IT IS MET IN RECOVERCHAIN
01	A single fixed case lifecycle	Nine ordered stages implemented as one <code>CasePipelineService</code> ; a persisted <code>CaseState</code>
02	Explainability at every stage	Each engine returns its signals, confidence, rule-by-rule results, and a plain-language line; all captured in <code>audit_records</code> .
03	ML strictly advisory (“shadow-only”)	<code>shadow_mode_active: true</code> is hard-coded in the predictor; no ML value is an input to any <code>PolicyDecision</code> ; missing/failed model → deterministic baseline.
04	Dataset-agnostic ingestion	Dataset Lab: format-agnostic reader (CSV/Parquet/XLSX), semantic column mapper, leakage detector, minimum-information contract, per-dataset training.
05	Real safety gate with real history	<code>DeterministicPolicyEngine</code> + <code>PolicyContext</code> built from the <code>execution_attempts</code> ledger; enforces retry/cooldown/contact/amount/consent/stop rules.
06	Honest execution & verification	<code>MockExecutionAdapter</code> only; idempotency keys; independent settlement reconciliation clamped to <code>[0, amount]</code> .
07	Offline, audited, demonstrable	Zero runtime external calls; full audit table; 216 passing tests; verified end-to-end in a browser.
08	Deployable and hardened	Auth on all mutating routes, rate limiting, CORS correctness, non-root container, CI with lint + types + coverage gate.

SECTION 2

Literature Review

RecoverChain draws on four strands of prior work and practice: dunning / involuntary-churn management, payment-failure prediction, safe deployment patterns for machine learning, and human-in-the-loop decision support.

INVOLUNTARY CHURN AND DUNNING MANAGEMENT

The subscription-economy literature and vendor practice (recovery and billing platforms, processor retry engines) establish involuntary churn as a first-class, recoverable revenue category and dunning — timed retries plus escalating customer messaging — as the standard mitigation. The consistent finding is that *timing* and *message sequencing* matter, and that naive fixed schedules leave money uncollected. RecoverChain accepts the dunning primitives (retry, reminder, payment-link, payment-method update, escalation) but subordinates them to a per-case diagnosis rather than a global schedule.

PAYMENT-FAILURE AND CHURN PREDICTION

Supervised models — gradient-boosted trees prominent among them — are widely used to estimate the probability that a given charge will fail or that a subscriber will lapse, using billing history, tenure, payment-method attributes, and prior-failure counts. Two recurring methodological hazards are emphasised in this literature: **target leakage** (post-outcome columns such as a settlement date or final status leaking the answer into the features) and **improper validation** (random splits on time-ordered data inflating scores). RecoverChain’s

Dataset Lab operationalises both defences: an explicit leakage detector that quarantines post-outcome columns, and a chronological train/validation/test split with a flagged stratified fallback.

SAFE ML DEPLOYMENT — SHADOW MODE

Work on the operational risks of production ML (notably the “hidden technical debt” framing from Sculley et al., 2015) motivates **shadow deployment**: running a model against live traffic while its outputs are logged and compared but never acted on. RecoverChain adopts shadow mode not as a transient rollout phase but as a *permanent architectural boundary* — the predictor cannot be promoted into the decision path without changing the type signature of the policy engine, and the model card states this explicitly.

RULES ENGINES VS. LEARNED POLICIES; HUMAN-IN-THE-LOOP

Decision-support research contrasts transparent rule systems (auditable, explainable, brittle to novelty) with learned policies (adaptive, opaque). For regulated financial actions the auditability requirement dominates. RecoverChain uses deterministic rules for every *decision*, reserves ML for *ranking*, and inserts an explicit human-escalation state (`ESCALATE` / human-review endpoint) for cases above the automated-amount ceiling or flagged by a stop rule — a conventional human-in-the-loop checkpoint.

POSITIONING

Where prior systems either schedule blindly or let a model decide, RecoverChain contributes a concrete architecture in which a **diagnosis-driven deterministic pipeline** makes every decision, a **ledger-backed policy gate** enforces safety against real history, and a **permanently shadowed model** supplies only prioritisation — all runnable offline with a full audit trail.

SECTION 3

Existing System

3.1 Traditional System

In current practice, recovery of failed revenue is handled by some combination of:

- **Processor-native retries.** The payment gateway re-attempts a failed charge on a built-in cadence (fixed intervals, or a statistically chosen “smart” time). Configuration is coarse — typically a retry count and window — and the retry has no view of *why* the charge failed.
- **Standalone dunning / recovery tools.** A SaaS product watches for failed invoices and sends a pre-authored email/SMS sequence (“your payment failed” → “reminder” → “final notice”), optionally with a hosted card-update page. Sequences are the same for every customer and every failure reason.
- **In-house scripts.** A finance or growth team writes cron jobs that query the billing database and fire retries or emails against hand-tuned thresholds. Logic accumulates as special cases; there is rarely an audit trail or a safety layer.

- **Manual collections.** For high-value B2B invoices, a human works a queue: calls, emails, escalations. Effective but unscalable, and the reasoning lives in the collector’s head.
- **Bolt-on ML scoring.** Some teams add a churn/failure model to prioritise the queue. When the model’s score is wired directly into “who gets retried / dunned”, it becomes an unaudited decision-maker.

3.2 Feature Comparison

CAPABILITY	PROCESSOR RETRIES	DUNNING SAAS	IN-HOUSE SCRIPTS	MANUAL COLLECTIONS	RECOVERCHAIN
Event → case correlation	—	Per-invoice	Ad hoc	Per-account	Yes — shared <code>reference_id</code>
Root-cause diagnosis	—	—	—	In collector’s head	Yes — 9 cause categories
Per-case action choice	Retry only	Fixed sequence	Threshold rules	Human judgement	Yes — ranked candidates + ERV
Recovery-probability estimate	—	—	Rare	Gut feel	Yes — shadow model or baseline
Safety gate (caps / cooldown / consent / stop)	Retry count only	Send limits	Partial	Policy in a doc	Yes — 8 rule types, ledger-backed
Independent outcome verification	—	Assumes send = done	—	Manual reconcile	Yes — settlement source, clamped
Full audit trail	Processor logs	Message logs	Rare	Notes	Yes — every transition
Bring-your-own dataset + model	—	—	—	—	Yes — Dataset Lab, per-dataset training
ML barred from deciding	n/a	n/a	Often not	n/a	Yes — architectural boundary
Runs offline	—	—	Varies	n/a	Yes

3.3 Limitations of the Existing System

- **Schedule, not diagnosis.** Retries and reminders fire on a calendar regardless of the failure reason, so effort is spent where it cannot work (expired card, rejected mandate, disputed invoice).
- **One cadence for everyone.** A ₹49 mobile top-up and a ₹9,800 B2B invoice get the same treatment.
- **No explanation.** When a customer is retried or emailed, there is no recorded reason, confidence, or expected value — nothing to review or appeal.
- **Weak or absent safety layer.** Contact-frequency and retry caps, cooldowns, consent, and hard stops are partial, per-tool, or written in a policy document rather than enforced in code against real history.
- **“Send equals recovered.”** Success is inferred from having sent a message or fired a retry, not from confirmed settlement.

- **Unaudited ML.** Where a model is used, its score often silently drives who gets actioned, with no boundary preventing it and no record of its influence.
- **Rigid data assumptions.** Each tool expects its own schema; feeding in a novel billing export or testing a hypothesis on historical data is not supported.

SECTION 4

Proposed System

RecoverChain is a two-part system: a **decisioning core** that runs every case through one auditable lifecycle, and a **Dataset Lab** that turns an arbitrary tabular file into scored cases — presented through a ten-step guided workflow and a gated report.

4.1 Feature Overview

DECISIONING CORE

- **Event ingestion & correlation.** Single (`/events`) or bulk (`/events/batch`) revenue-failure events, deduplicated by `external_event_id` and correlated into cases by `reference_id`.
- **Risk assessment.** A bounded score (amount curve $1-e^{-\text{amount}/800}$ combined with retry / age / churn sub-scores per category) mapped to `LOW / MEDIUM / HIGH / CRITICAL`, with the contributing signals returned.
- **Root-cause diagnosis.** Evidence from the linked events is matched to nine cause categories (`INSUFFICIENT_FUNDS`, `PAYMENT_METHOD_INVALID`, `NETWORK_FAILURE`, `MANDATE_FAILURE`, `UNRESOLVED_DISPUTE`, `MISSED_COMMITMENT`, `PAYMENT_FRICTION`, `CONFLICTING_EVIDENCE`, `UNKNOWN`) with a confidence and `CONFIRMED / INFERRED / UNKNOWN` status.
- **Recovery prediction.** A probability from a per-dataset shadow model when one exists and passes the quality gate, otherwise from `DeterministicBaselinePredictor` — never a silent zero. Always labelled `SHADOW_ONLY`.
- **Action recommendation + ERV.** Candidate actions are generated for the diagnosed cause, scored, and ranked; the top candidate carries an **expected recoverable value** (amount × probability factor).
- **Policy & safety gate.** `DeterministicPolicyEngine` returns `PERMITTED / WAIT / ESCALATE / DENIED` after evaluating stop, consent, evidence, human-escalation, automated-amount, max-retries, retry-cooldown, and contact-frequency rules — counting *real* attempts from the ledger when a `PolicyContext` is supplied.
- **Sandboxed execution.** `AgentOrchestrator` dispatches the permitted action to a specialist agent through `MockExecutionAdapter`; requires a fresh matching `PERMITTED` decision; idempotent via `idempotency_keys`; writes an `execution_attempts` ledger row.
- **Independent verification.** `VerificationEngine` reconciles against a `SANDBOX_SIMULATION` settlement source; the actual recovered amount comes only from the `RecoveryOutcome` and is clamped to `[0, amount]`.
- **Audit.** Every state transition writes an `audit_records` row with from/to state and an evidence blob.

- **Observability.** Read-only `/system/*` endpoints expose cases, events, models, executions, policy decisions, audit, and a CSV export.

DATASET LAB

- **Upload** CSV / Parquet / XLSX / ZIP (size-capped, path-sanitised, auth-guarded).
- **Profiling & semantic mapping.** Columns are profiled and mapped to canonical fields (`ENTITY_ID`, `AMOUNT`, `TIMESTAMP`, `TARGET`, ...) with confidence; the pipeline pauses at `MAPPING_REVIEW` for user confirmation and validates overrides server-side.
- **Quality & leakage.** A data-health score, completeness breakdown, dead-column check, and a leakage detector that quarantines post-outcome columns.
- **ML-readiness contract.** Derives target, feature and excluded columns (identifier / constant / post-outcome / high-cardinality, each with a reason), the temporal split strategy, and class balance; blocks datasets failing the minimum-information contract.
- **Per-dataset training.** An sklearn `Pipeline` (text→numeric/epoch coercion, impute, scale, one-hot, LogReg + XGBoost candidates, calibration) with a chronological split and a ROC-AUC ≥ 0.55 quality gate; failing models are stored `REJECTED_LOW_QUALITY`.
- **Replay.** `generate-cases` reads rows, auto-trains a model, and runs each row through the full pipeline — producing real, inspectable cases with honest accepted/skipped counters.
- **Registry hygiene.** Dataset pruning that also sweeps orphaned model artifacts; strict dataset↔model isolation by `dataset_id`.

PRESENTATION

SURFACE	ROUTE	WHAT IT IS
RecoverChain workflow	<code>/</code>	A ten-step guided flow, one full-screen step at a time, <code>Proceed →</code> navigation, plain-language copy, honest about real vs. simulated. Optional lazy-loaded 3D / motion backdrop, <code>prefers-reduced-motion</code> -safe.
Insights report	<code>/insights</code>	Plain-language aggregation over every recovery run — money at risk, recovered, recovery rate, outcome mix, breakdown by cause and by action. Locked until the workflow has been completed once.

4.2 Technology Stack

LAYER	TECHNOLOGY	ROLE
Backend	Python 3.12 (dev venv 3.14)	Runtime; CI and container pinned to 3.12.
	FastAPI + Starlette + Uvicorn	HTTP API, dependency-injected routes, ASGI server.
	Pydantic v2	Domain models, request/response schemas, validation.
	SQLAlchemy 2.0 + Alembic	ORM and migrations; migrations own the Postgres schema.
	SQLite / PostgreSQL	SQLite is the zero-setup offline default; Postgres for deployment.
	Redis (optional)	Only <code>/health</code> touches it; system runs without it.
Machine learning	scikit-learn	Pipeline, preprocessing, LogisticRegression, calibration, metrics.
	XGBoost	Gradient-boosted candidate model.
	pandas · NumPy · pyarrow · openpyxl	Format-agnostic dataframe I/O and feature coercion.
Frontend	React 18 + Vite 5 + TypeScript	Single-page app, fast dev/build.
	react-router v6	Workflow / Insights routing.
	Tailwind CSS v3 (PostCSS <code>.cjs</code>)	Utility styling.
	three · @react-three/fiber · @react-spring/three · GSAP	Lazy-loaded, toggleable motion backdrop.
Tooling / Ops	pytest + pytest-cov	216 tests; 80% coverage gate in CI.
	Ruff + Mypy	Lint, format, typed-core checking.
	ESLint + Prettier	Frontend lint and format.
	Docker Compose + GitHub Actions + Dependabot	Postgres/Redis/api/web images; CI runs lint, types, migrations round-trip, tests+coverage, audits, image builds.

4.3 Benefits over Existing Solutions

- **Effort follows the diagnosis.** A case caused by an expired card is routed to a card-update request, not a doomed retry; a disputed invoice is escalated, not reminded.
- **Every output is defensible.** Risk, cause, probability, action, and value each ship with their signals and confidence; the policy verdict is rule-by-rule; all of it is in the audit table.
- **Safety is enforced, not documented.** Retry caps, cooldowns, contact limits, an amount ceiling, consent, and stop rules are code, evaluated against the real attempt ledger.
- **Outcomes are measured, not assumed.** Recovered amounts come only from an independent settlement check.
- **ML cannot misfire.** The model is structurally unable to authorise an action; absence or low quality degrades gracefully to a deterministic baseline.
- **Any dataset, safely.** The Dataset Lab makes historical experimentation a first-class feature, with leakage and sufficiency checks built in.

- **Runs anywhere, needs nothing.** No CDN, no external API, Redis optional — demonstrable on a laptop with no network.

SECTION 5

Concept & Methodology

5.1 Guiding concepts

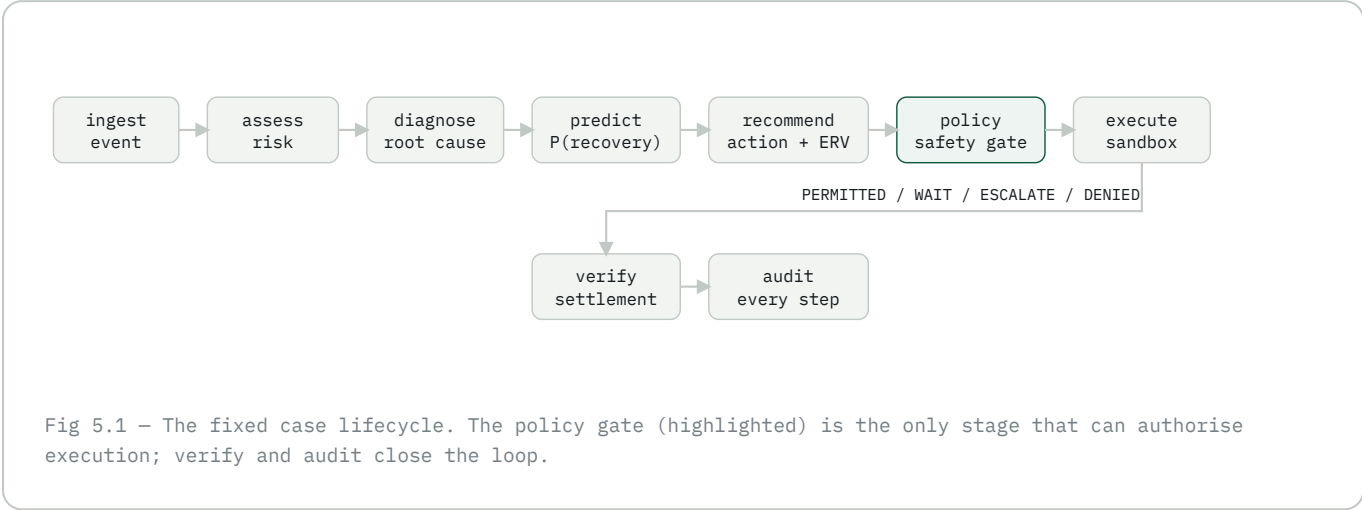
- **The case as the unit of work.** Individual events are noisy and duplicative; the `RecoveryCase` aggregate is what carries state, accumulates the seven stage sub-objects, and is persisted and reasoned about.
- **One lifecycle, always the same order.** There is exactly one path through the stages. This makes the system predictable, testable stage-by-stage, and auditable — the sequence of `audit_records` for a case is a complete, ordered story.
- **Determinism for decisions, statistics for priority.** Anything that changes what the system *does* is a pure function of the case and explicit policy. Anything learned from data only changes the *order* in which humans or the queue see cases.
- **Shadow mode as a boundary, not a phase.** The predictor returns `shadow_mode_active: true` and its value is never a parameter to policy evaluation. Promoting it would require changing a type signature — a deliberate, reviewable act.
- **Honesty in the UI.** Every screen states whether a number is a rule output, a shadow-model estimate, or a simulation; execution and settlement are labelled `SANDBOX_SIMULATION`.
- **Offline-first.** A hard constraint that shaped dependency choices (bundled fonts, no CDN, Redis optional, a hand-rolled `StateGraph` where a library needed a blocked native module).

5.2 Software methodology

- **Clean / layered architecture (DDD-influenced).** `domain/` is framework-free Pydantic models and the state machine; `application/` is one engine per stage plus orchestration; `infrastructure/` is ORM, repositories, adapters; `api/` is thin FastAPI routers. Dependencies point inward only.
- **Test-first, invariant-driven.** The safety invariants (ML shadow-only, execution simulated, policy is sole authority, financial clamping, dataset↔model isolation, migrations own the schema, auth on mutating routes) each have dedicated regression tests. The suite runs against an isolated throwaway SQLite database.
- **Iterative, phase-by-phase delivery.** The project advanced through ~24 numbered build phases (architecture notes for phases 4–23 are kept under `docs/history/`) and 33 commits, each a self-contained, test-green increment — from “make the live case loop flow” through “ledger-backed policy”, “ML quality gate”, “offline”, the 10-step workflow redesign, real-data training, a performance fix, and the security/tooling hardening pass.
- **Reproducible ML.** Training is a single scripted entry point (`run_phase19_training.py`); models are written to a registry with metadata (dataset id, split counts, metrics, status) and a model card documents intended use and limitations.

- **Environment-driven configuration.** Behaviour (database URL, API key, CORS list, rate limits, upload cap, quality-gate thresholds, currency) is set by environment variables with safe offline defaults and a startup warning when insecure defaults are left in place.

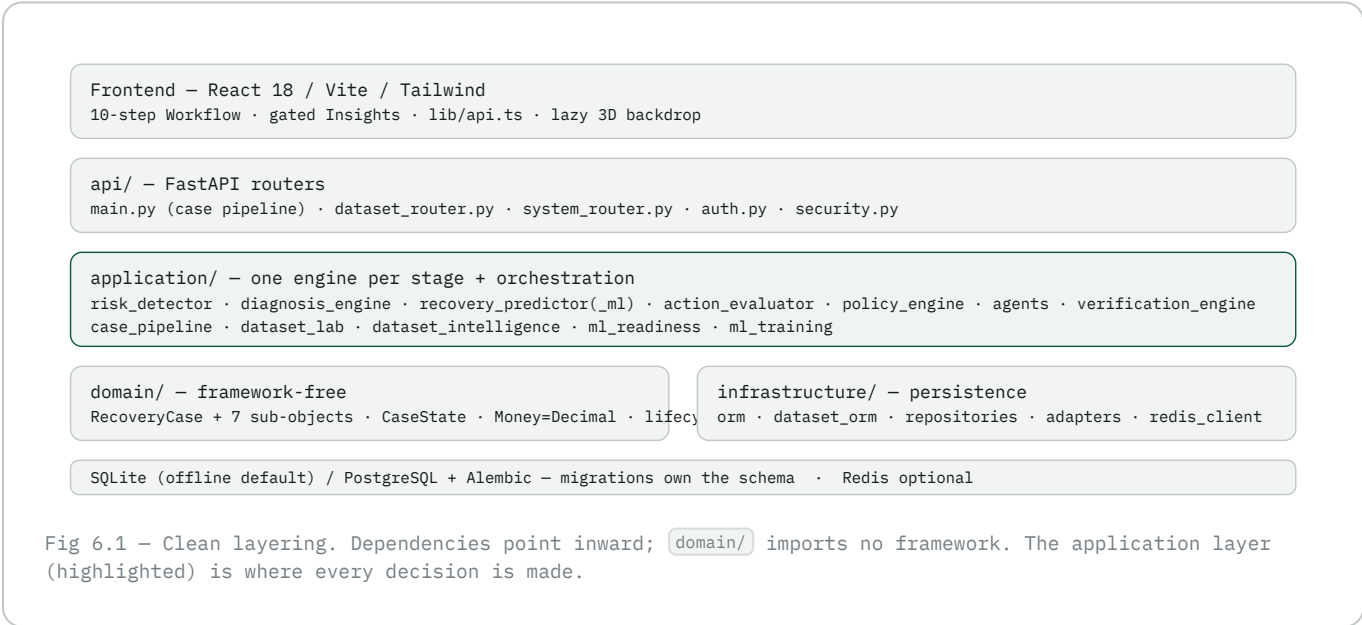
5.3 The lifecycle, end to end



SECTION 6

Design, Modelling & System Architecture

6.1 Layered architecture



6.2 Domain model

The aggregate root is `RecoveryCase`. It holds identity and money fields, a `current_state`, the list of linked events, and seven optional stage sub-objects populated as the case advances:

SUB-OBJECT	PRODUCED BY	KEY FIELDS
RiskAssessment	risk_detector	score (0–1), risk_level, primary_risk_signals, detection_status
RootCauseDiagnosis	diagnosis_engine	cause_category (9), confidence, status, supporting_signals
RecoveryPrediction	recovery_predictor(_ml)	recovery_probability, model_version, prediction_status (SHADOW_ONLY)
ActionRecommendation	action_evaluator	candidates[], top_candidate, status, rationale
PolicyDecision	policy_engine	status, rules_evaluated[], failed_rules[], reason
ExecutionRecord	agents	action_type, agent_type, status, adapter_used, result_metadata
RecoveryOutcome	verification_engine	status (4), actual_amount_recovered, verification_source, reconciliation_status

Money is a Pydantic model wrapping a `Decimal` amount plus a currency (default `INR`); the ORM stores `Numeric(18,4)`. **Enumerations** are all string enums: `RiskCategory` (5), `RiskLevel` (4), `RootCauseCategory` (9), `ActionType` (~17), `PolicyDecisionStatus` (4), `ExecutionStatus` (5), `RecoveryOutcomeStatus` (4).

CASE STATE MACHINE

`CaseState` has 21 members spanning the lifecycle: `DETECTED` → `OPEN` → `ASSESSED` → `DIAGNOSING` → `RECOMMENDING` → `POLICY_EVALUATED`, then a branch — `APPROVED` / `DENIED` / `WAITING` / `ESCALATED` / `POLICY_REVIEW` — then `EXECUTION_PREPARING` → `EXECUTING` → `PENDING_VERIFICATION` → `VERIFYING`, terminating in `FULLY_RECOVERED` / `PARTIALLY_RECOVERED` / `CLOSED_NOT_RECOVERED` or the out-of-band `STOPPED`. Every transition is written to `audit_records` with its true prior state.

6.3 Persistence schema

Seven tables; the lifecycle sub-objects are stored as JSON columns on `recovery_cases` (a deliberate trade-off — fast to evolve, noted as future relational work). Migrations own the Postgres schema; SQLite auto-creates for local dev.

TABLE	PURPOSE	NOTABLE COLUMNS
recovery_cases	Case aggregate	case_id, customer_id, reference_id, amount_at_risk, current_state, + 7 JSON sub-object columns
revenue_events	Correlated raw events	event_id, external_event_id, reference_id, case_id, amount, raw_payload
audit_records	Every state transition	case_id, from_state, to_state, evidence (JSON), timestamp
execution_attempts	Ledger — source of policy history	case_id, action_type, idempotency_key, status, created_at
idempotency_keys	Replay protection	key, execution_id
datasets	Dataset Lab registry	dataset_id, status, columns_profile, recoverchain_signals, data_quality_report, leakage_detection, training_suitability

6.4 API surface

GROUP	ENDPOINTS
Ingestion	POST /events, POST /events/batch
Case pipeline	POST /cases/{id}/advance, .../stop, .../{assess-risk,diagnose,predict-recovery,recommend-action,policy-check,execute,verify}, .../human-review
Case reads	GET /cases, /cases/{id}, /cases/{id}/{events,audit}, /dashboard/metrics
Dataset Lab (writes)	POST /datasets/{upload, sync, prune, {id}/analyze, {id}/ml-readiness, {id}/train, {id}/mapping, {id}/predict, {id}/generate-cases}
Dataset Lab (reads)	GET /datasets, /datasets/{id}, /datasets/{id}/{preview,workflow-status,ml-readiness,models}
Observability	GET /system/{health,summary,cases,cases.csv,cases/{id},datasets,models,events,policy,executions,audit}
Evaluation	POST /evaluation/run, GET /health

6.5 Dataset Lab state machine

```
# explicit, review-driven – the pipeline pauses for the user at MAPPING_REVIEW
UPLOADED → PROFILING → MAPPING_REVIEW → READY_FOR_ANALYSIS
    → ANALYZING → ML_READY → TRAINING → TRAINED
    → (REJECTED_LOW_QUALITY → deterministic baseline)
```

6.6 Security architecture

- **Authentication.** `verify_api_key` (X-API-Key header) guards every mutating route on both the case pipeline and the Dataset Lab; read-only GETs stay open for the dashboard. A startup check warns when the dev-default key or wildcard CORS is left in place.
- **Rate limiting.** A stdlib sliding-window limiter (per key / per IP, default 120 req/min) as ASGI middleware; health and docs are exempt.
- **CORS.** Wildcard origin and credentialed requests are mutually exclusive by spec — the app enables credentials only when an explicit allow-list is configured.
- **Uploads.** Size-capped (`MAX_UPLOAD_BYTES`, 512 MiB default), extension-checked, filename path-sanitised, dataset-scoped storage names.
- **Container.** Non-root user, health-check, migrations must succeed before serving. Compose requires explicit secrets rather than shipping defaults.
- **Frontend.** The API key is build-time config (`VITE_API_KEY`), not a source constant; nginx sets CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, and gzip.

Implementation

7.1 Stage engines (application/)

RISK_DETECTOR

Computes a bounded amount score $1 - e^{-\text{amount}/800}$ and combines it with a category-specific sub-score — retry count for failed payments/subscriptions, churn risk for checkout abandonment, age-in-hours for overdue invoices and broken promises — then maps the blended 0–1 score to `LOW <0.3`, `MEDIUM <0.6`, `HIGH <0.85`, `CRITICAL`, returning every signal used.

RECOVERY_PREDICTOR / _ML

`DeterministicBaselinePredictor` gives a rules-based probability from cause and risk.
`MLPaymentFailurePredictor` loads only a model whose metadata `dataset_id` matches exactly and whose status is not `REJECTED_LOW_QUALITY`; recovery probability = $1 - \text{failure_risk}$. Both stamp `SHADOW_ONLY`.

POLICY_ENGINE

`DeterministicPolicyEngine.evaluate(case, context)` runs, in order: `StopRule`, `ConsentCheck`, `EvidenceCheck`, `HumanEscalationAction`, `FinancialAutomatedLimit` (₹5,000 default ceiling), `PaymentMaxRetries` (3), `PaymentRetryCooldown` (24 h), `CommunicationMaxMessages` (2 / 24 h). With a `PolicyContext` the retry/cooldown/contact counts come from the real ledger; without one it falls back to a linked-events proxy for unit tests. Result: `PERMITTED` / `WAIT` / `ESCALATE` / `DENIED` with per-rule detail.

`case_pipeline.py`'s `CasePipelineService` runs assess → diagnose → predict → recommend → policy in one call and is the shared path for `/events/batch`, `/cases/{id}/advance`, and `generate-cases`.

7.2 Dataset Lab (application/)

- `dataset_lab.py` — registry service: import, list, prune; file storage under dataset-scoped names.

DIAGNOSIS_ENGINE

Deterministic evidence matching from linked-event payloads to nine `RootCauseCategory` values, emitting a confidence and a `CONFIRMED` / `INFERRED` / `UNKNOWN` status plus supporting signal references. Receivables vocabulary (`net_terms_pending`, `dispute`, `bank_rejected` ...) maps onto the shared cause set.

ACTION_EVALUATOR

Generates candidate `ActionType`s appropriate to the diagnosed cause and category, scores and ranks them, and attaches an **expected recoverable value** to the top candidate ($\text{amount} \times \text{probability factor}$). Returns `RECOMMENDED` or `INSUFFICIENT_EVIDENCE`.

AGENTS / VERIFICATION_ENGINE

`AgentOrchestrator.execute` requires a persisted, fresh, matching `PERMITTED` decision and a capable specialist agent, dispatches through `MockExecutionAdapter`, and (given a repo) writes the idempotency key + ledger row.
`VerificationEngine.reconcile` checks the `SANDBOX_SIMULATION` settlement source, sets the outcome status, and clamps the recovered amount to `[0, amount]`.

- `dataset_intelligence.py` — `SemanticMapper` normalises column names, inspects values, and scores each column against canonical fields with `HIGH / MEDIUM / LOW` confidence; ambiguous single-use fields (AMOUNT, TARGET, TIMESTAMP...) are downgraded and flagged for user confirmation. `DatasetValidator` runs leakage detection and dataset classification (`ANALYSIS_READY` ... `INSUFFICIENT`).
- `ml_readiness.py` — derives target, feature columns, excluded columns with reasons, temporal-split strategy, and class balance; enforces the minimum-information contract.
- `ml_training.py` — the training pipeline (below).

ML TRAINING PIPELINE

1. **Format-agnostic read** — `_read_any()` branches on extension (Parquet / Excel / CSV).
2. **`_coerce_frame` step** — turns number-like text (`"1,200"` , `"₹ 45"`) into floats and date-like text into epoch-day floats, *at fit and predict time*, so a messy real CSV transforms identically when served.
3. **Preprocessing** — impute + scale numerics; one-hot categoricals with `handle_unknown="ignore"`.
4. **Candidates** — `LogisticRegression` and `XGBClassifier`, probability-calibrated on the validation fold (skipped and folded back into train when that fold is tiny or single-class).
5. **Split** — chronological on the real date column; **stratified fallback** (flagged in metadata) when there is no usable temporal column.
6. **Quality gate** — ROC-AUC $\geq$ `ML_MIN_ROC_AUC` (0.55) and rows $\geq$ `ML_MIN_TEST_ROWS`; an adaptive mode lowers the row floor for small real datasets. Failing models are written `REJECTED_LOW_QUALITY` and are never loaded.
7. **Registry** — `{model.joblib, metadata.json}` keyed by dataset id, with split counts, metrics, and status.

7.3 Infrastructure

- `repositories.py` — `SQLAlchemyCaseRepository` maps ORM $\leftrightarrow$ domain, and provides `record_execution_attempt` and `get_policy_context` (which builds a `PolicyContext` from the `execution_attempts` ledger). Commits per use-case.
- `orm.py` / `dataset_orm.py` — the seven tables; lifecycle sub-objects as JSON columns; timezone-aware timestamps.
- `adapters.py` — `MockExecutionAdapter` and the settlement source; the only “external world” and both simulated.
- `langgraph_orchestrator.py` — a hand-rolled `StateGraph` used only by the evaluation harness, because the real library needs a native module blocked in the target environment. Kept deliberately dependency-free.

7.4 API layer

Thin FastAPI routers. `main.py` carries the case-pipeline endpoints, batch ingest, advance/stop, a request-id + timing middleware, and the dashboard aggregation. `dataset_router.py` and `system_router.py` carry the Dataset Lab and read-only observability. `auth.py` and `security.py` hold the API-key dependency, the rate limiter, the startup config check, and the upload-size constant.

7.5 Frontend

#	STEP	SHOWS (ALL FROM LIVE API DATA)
01	Upload data	Dropzone or bundled sample; real accepted formats.
02	Data review	Real filename, row/column counts, detected canonical fields.
03	Data quality	Health score, completeness breakdown, leakage check.
04	Data mapping	The four roles pre-filled from detection; low confidence flagged; server-validated on confirm.
05	Data connection	How the file wires into the pipeline — canonical inputs, model features vs. held-out columns with reasons, split strategy, class balance, readiness verdict; honest note that multi-file linking is not built.
06	Revenue risk detection	Runs <code>generate-cases</code> ; real accepted / skipped counters.
07	AI analysis	Highest-value case: cause + confidence, recovery probability (advisory), expected recoverable value; rule-based vs. shadow-model labelled.
08	Decision & policy	Merged — the rules-engine proposal <i>and</i> the policy engine's rule-by-rule verdict on one screen.
09	Recovery	<code>PERMITTED</code> → executed (simulated); escalated → “Approve & execute”; blocked → not executed, with the reason.
10	Verified result	Outcome verified against the sandbox settlement source; actual amount recovered.

Reaching step 10 sets a `localStorage` flag that unlocks the **Insights** report. `lib/api.ts` is the single API wrapper; the motion backdrop (Three.js / R3F / GSAP) is a lazy-loaded chunk, toggleable and reduced-motion-safe.

7.6 Hardening & tooling pass

A dedicated pass addressed operational readiness: API-key auth extended to the entire Dataset Lab; a request rate limiter; CORS credential/wildcard correctness; a startup insecure-default warning; upload-size enforcement made configurable; the Gemini evaluation key moved from URL to header; dead dependencies removed and a release-candidate pin dropped; a non-root, migrate-then-serve container; nginx security headers; `pyproject.toml` with Ruff + Mypy + coverage config; a `requirements-dev.txt`; ESLint + Prettier and stricter `tsconfig` for the frontend; a rewritten CI (lint, format, types, Alembic round-trip, tests + 80% coverage gate, `pip-audit`, `npm audit`, Docker image builds, branch filter, concurrency) and a Dependabot config; and repository hygiene (≈55 one-off scripts archived out of the root). All 216 tests remained green throughout.

7.7 Code metrics

8,332 LOC · APP/API/DOMAIN/INFRA	6,080 LOC · BACKEND TESTS	3,665 LOC · FRONTEND TS/TSX/CSS
42 TEST FILES	7 DB TABLES	~40 HTTP ENDPOINTS

SECTION 8

Results & Discussion

8.1 Automated test suite

216 backend tests pass against an isolated throwaway SQLite database; line coverage is 88% (CI gate 80%). Coverage by area:

AREA	TESTS	REPRESENTATIVE FILES
Stage engines (risk, diagnosis, prediction, recommendation, policy, execution, verification)	70	test_risk_detector, test_diagnosis (17), test_policy (9), test_verification
Ingestion, correlation, case engine	32	test_phase28_ingestion (20), test_correlation (8), test_case_engine
Dataset Lab & ML	44	test_dataset_intelligence, test_ml_readiness, test_ml_training, test_ml_quality_gate, test_shadow
API, observability, auth, hardening	40	test_backend_observability (12), test_auth (3), test_phase32_hardening (5)
End-to-end workflow & safety regression	18	test_phase27_e2e_integration, test_phase29_e2e_universal_workflow, test_phase34_1_safety_regression
Domain, contracts, evaluation, roadmap features	12	test_domain, test_inference_contract, test_roadmap_track_a

8.2 End-to-end verification in a browser

The full stack was run (SQLite backend on :8000, Vite on :5173) and the ten-step workflow driven through with the bundled sample dataset. Every step produced live data and every guarded route returned 200 with the frontend supplying the API key:

STEP	OBSERVED RESULT	ROUTE EXERCISED
Upload sample	15 rows / 5 columns read; columns auto-mapped	POST /datasets/upload
Data quality	health 100/100; leakage check clean	POST /datasets/{id}/analyze
Confirm mapping	accepted after server-side validation	POST /datasets/{id}/mapping
Risk detection	13 recovery cases created from 15 rows (2 skipped — already paid / nothing to recover)	POST /datasets/{id}/generate-cases
AI analysis	case 68914584 : ₹15,000 at risk, cause “insufficient funds” (95% conf.), recovery probability 67% (advisory), ERV ₹10,110	pipeline
Decision & policy	proposed “Retry the payment”; policy flagged FinancialAutomatedLimit — ₹15,000 > ₹5,000 ceiling → needs human approval	policy engine
Recovery	human approval gate shown; on approve → executed via MockExecutionAdapter (“Payment retried successfully”)	POST /cases/{id}/human-review + /execute
Verified result	Not recovered , ₹0.00 recovered, reconciliation “confirmed failure from source” (SANDBOX_SIMULATION)	POST /cases/{id}/verify
Insights (unlocked)	49 cases across 12 datasets, ₹43,827.50 at risk, ₹9,947.34 recovered, 23% recovery rate, full outcome + cause + action breakdown	/dashboard/metrics, /cases, /system/*

Zero console errors; no 500 s; clean backend log. The escalate-then-execute-then-fail-verification path is exactly the intended behaviour: an executed action is *not* assumed to have recovered money — the settlement check is authoritative.

8.3 Security behaviours, verified live

```
POST /datasets/prune (no key) → 403
POST /datasets/sync (bad key) → 403
POST /datasets/sync (good key) → 200
130× rapid GET /cases → 120×200 then 10×429 # 120 req/min budget, exact
```

8.4 Machine-learning behaviour

- A ~140-row engineered dataset auto-trains an XGBoost model (ROC-AUC $\approx$ 1.0 on its held-out fold) that then scores its cases SHADOW_ONLY.
- Noisier small datasets correctly fail the 0.55 gate, are stored REJECTED_LOW_QUALITY, and the pipeline serves the DeterministicBaselinePredictor instead — never a silent 0.0.
- In neither case does any probability enter a PolicyDecision; a dedicated regression test asserts the boundary.

8.5 Performance

An early defect made Step 06 appear to hang: predict_recovery_for_case() rebuilt a full sklearn+XGBoost predictor (scanning a ~400-file registry, joblib.load each call) **once per row**. The fix threads an optional pre-built predictor through the pipeline so

`generate-cases` constructs it **once**.

PREDICTOR INITIALISATIONS		ENDPOINT TIME (77-CASE RUN)
Before	78	3.23 s
After	1	2.65 s

The saving scales with row count and registry size; a regression test asserts exactly one construction per request. Frontend build: main bundle ≈ 316 kB (≈ 104 kB gzip); the Three.js backdrop chunk ≈ 859 kB (≈ 236 kB gzip) is lazy-loaded.

8.6 Discussion

What works. The deterministic pipeline is genuinely explainable end to end — a case’s audit trail reconstructs every decision. The policy gate behaves correctly against real ledger history (the ₹15,000 case escalated exactly as configured). The shadow-only boundary held under test and in the live run. The Dataset Lab turns an arbitrary CSV into inspectable cases with leakage and sufficiency checks that actually fire.

Honest limitations. Execution and settlement are simulated — there is no live payment or communications adapter. The domain is still partly JSON-blob rather than fully relational, so cross-case querying is limited. Correlation needs a shared `reference_id`; there is no entity resolution across files yet. The dashboard aggregates in application code rather than SQL. The frontend has no automated tests. There is no multi-tenancy. The reference model’s training data provenance is unverified (documented in the model card). These are scoped in Section 10.

SECTION 9

Conclusion

RecoverChain demonstrates that failed-revenue recovery can be treated as a transparent decisioning problem rather than a blind schedule. Each failure becomes a case; each case runs one fixed, auditable lifecycle of small deterministic engines; a ledger-backed policy gate is the sole authority over whether an action runs; execution is sandboxed and idempotent; and the outcome is verified independently rather than assumed. Machine learning is present and useful for prioritisation, but is architecturally prevented from ever deciding — and degrades cleanly to a deterministic baseline when it is absent or weak.

The Dataset Lab extends the same discipline to arbitrary data: any tabular file can be mapped, checked for leakage and sufficiency, used to train a per-dataset model, and replayed into real cases — still without that model gaining authority.

The system is validated by 216 passing tests at 88% coverage, a clean end-to-end browser walk-through, and a hardening pass that makes it safe to deploy and demonstrate. It runs entirely offline. Within its deliberately simulated execution boundary, RecoverChain is a complete, coherent, and defensible implementation of diagnosis-driven revenue recovery.

SECTION 10

Future Scope

TRACK	ITEM	DESCRIPTION	STATE
Near-term	Trust & explainability	Optional LLM narrator (behind a key, off by default) replacing template lines; CSV/XLSX export of cases and outcomes; per-dataset and trend breakdowns in Insights.	partial
	Workflow robustness	Persist run state across refresh; a case picker on Step 7; background <code>generate-cases</code> as a tracked job with progress.	planned
	Test & quality	Frontend test suite (the 10-step flow, column mapping, <code>computeInsights</code>); correlation edge cases; frontend build+tests in CI.	partial
Structural	First-class domain model	Customer, Transaction/Payment, Invoice, Subscription, CheckoutSession, PromiseToPay as relational entities; entity resolution + shared <code>reference_id</code> ; move queryable sub-objects out of JSON columns; SQL-side dashboard aggregation.	planned
	Multi-file & relationships	Join payments ↔ customers ↔ invoices on a shared key; relationship detection in the Dataset Lab; make Step 5 functional.	planned
Production	Real execution	Real payment adapter(s) (e.g. Stripe, Razorpay) and comms adapter (email/SMS) behind the existing interface; provider webhooks as settlement truth.	planned
	Multi-tenancy & hardening	Tenant isolation of data, models, and policy config; per-tenant policy (caps, cooldowns, thresholds, consent); production observability; distributed rate limiting; compliance and penetration testing.	planned

Milestone 1 of the near-term track (Excel/Parquet training, per-row risk category, configurable currency, payload sanitisation) and part of the tooling milestone (Ruff, Mypy, coverage gate, ESLint/Prettier, Docker build in CI) are **already delivered** in the current build.

SECTION 11

References

1. D. Sculley et al., “Hidden Technical Debt in Machine Learning Systems,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2015. — motivation for permanent shadow-mode boundaries.
2. T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” *Proc. 22nd ACM SIGKDD*, 2016. — the gradient-boosted candidate model.
3. F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, 2011. — pipeline, preprocessing, calibration, metrics.
4. S. Ramírez, *FastAPI Documentation*, 2018–present. fastapi.tiangolo.com — dependency-injected routing, security utilities.
5. M. Bayer, *SQLAlchemy 2.0 Documentation* and *Alembic Documentation*. docs.sqlalchemy.org — ORM and migration model.
6. Pydantic, *Pydantic v2 Documentation*. docs.pydantic.dev — domain modelling and validation.

7. C. Molnar, *Interpretable Machine Learning*, 2nd ed., 2022.
christophm.github.io/interpretable-ml-book — leakage, validation, explanation.

8. Stripe, “Smart Retries and automated failed-payment recovery,” *Stripe Billing Documentation*. — reference practice for processor-native retries and dunning.

9. Adyen, “RevenueProtect / auto-rescue,” *Adyen Documentation*. — reference practice for statistically-timed retries.

0. “Involuntary churn” and dunning benchmarks, subscription-analytics practitioner literature (ProfitWell / Recurly / Chargebee reports). — framing of involuntary churn as a recoverable revenue category.

1. React and Vite documentation. react.dev, vitejs.dev — the SPA and build tooling.

2. OWASP, *API Security Top 10* (2023) and *Secure Headers Project*. — auth, rate limiting, CORS, and header hardening guidance.

SECTION 12

Appendix

A Configuration (environment variables)

VARIABLE	DEFAULT	PURPOSE
DATABASE_URL	sqlite:///./recoverchain.db	SQLite offline default; Postgres for deployment (then alembic upgrade head).
API_KEY	test-api-key	X-API-Key on mutating routes; dev default logs a startup warning.
CORS_ORIGINS	*	Comma-separated allow-list; * disables credentialed CORS.
RATE_LIMIT_ENABLED / RATE_LIMIT_RPM	1 / 120	Per key/IP sliding-window request limit.
MAX_UPLOAD_BYTES	536870912	Dataset upload cap (512 MiB).
DEFAULT_CURRENCY	INR	Used when a dataset has no currency column.
ML_MIN_ROC_AUC / ML_MIN_TEST_ROWS	0.55 / 200	Training quality gate; 0 disables.
REDIS_URL	redis://localhost:6379/0	Optional; only /health uses it.
ALLOW_INSECURE_DEFAULTS	—	1 silences the insecure-default startup warnings.

B Commands

```
# full stack - backend :8000 + frontend :5173 (SQLite)
python run.py

# backend
cd backend && PYTHONPATH=. uvicorn api.main:app --reload
cd backend && PYTHONPATH=. pytest                # 216 tests
cd backend && PYTHONPATH=. pytest --cov          # coverage (CI gate 80%)
cd backend && ruff check . && ruff format --check .
cd backend && PYTHONPATH=. alembic upgrade head   # Postgres only
cd backend && PYTHONPATH=. python run_phase19_training.py

# frontend
cd frontend && npm install && npm run dev
cd frontend && npm run build && npm run lint && npm run format:check

# containers (copy .env.example -> .env first)
docker-compose up --build
```

C Repository layout

```
backend/
  domain/           # Pydantic models, CaseState, lifecycle, interfaces
  application/      # stage engines + case_pipeline + Dataset Lab + ml_*
  infrastructure/   # orm, dataset_orm, repositories, adapters, redis_client
  api/              # main.py, dataset_router.py, system_router.py, auth.py, security.py
  alembic/          # migrations (own the Postgres schema)
  evaluation/       # deterministic scenario harness + dataset metadata
  tests/            # 42 files, 216 tests
frontend/src/
  workflow/         # 10 step components + Rail, Frame, Backdrop
  pages/            # Insights, Caseslist, CaseView
  lib/              # api, format, motion, insights, progress
docs/
  history/          # architecture_phase*.md (phases 4-23)
  MODEL_CARD.md · ROADMAP.md · PROJECT_REPORT.md
scripts/legacy/     # archived one-off build scripts
```


D Glossary

TERM	MEANING
Case	The <code>RecoveryCase</code> aggregate — one account’s correlated failure events plus all lifecycle sub-objects and state.
Shadow-only	A model whose output is logged and surfaced for ranking but architecturally barred from authorising an action.
ERV	Expected Recoverable Value — <code>amount × probability factor</code> attached to the top recommended action.
Policy gate	<code>DeterministicPolicyEngine</code> — the sole authority over whether an action executes; returns <code>PERMITTED / WAIT / ESCALATE / DENIED</code> .
PolicyContext	Retry / cooldown / contact counts built from the real <code>execution_attempts</code> ledger.
Leakage	A post-outcome column (settlement date, final status...) that would let a model “see the answer”; quarantined by the Dataset Lab.
Minimum-information contract	The set of canonical fields a dataset must supply (entity, amount, timestamp, target) before it can train.
<code>SANDBOX_SIMULATION</code>	The settlement source used by verification; there is no live reconciliation.